

飞梭IOS-SDK接入文档（新）

- 一、概述
- 二、SDK 集成
 - 2.1 开发环境
 - 2.2 集成方式
 - 方式一：CocoaPods
 - 方式二：手动集成
 - 2.3 注意事项
- 三、SDK 初始化及全局配置
 - 3.1 说明
 - 3.2 接口说明
 - 3.3 接入示例
- 四、广告位与广告对象公共方法
 - 4.1 广告位对象（FSAdSlot）
 - 4.2 广告对象创建
 - 4.3 广告类型说明
 - 4.4 广告对象公共方法（FSUnionAd）
 - 4.4.1 公共方法
 - 4.4.2 使用示例
 - 4.4.3 FSBiddingLoseReason 竞价失败原因
 - 4.4.4 FSBiddingWinSDKType 竞胜方 SDK 类型
 - 4.4.5 FSAdInteractionType 枚举
- 五、开屏广告
 - 5.1 使用说明
 - 5.2 创建广告对象、请求广告
 - 5.3 展示广告
 - 5.4 展示时机
 - 5.5 回调说明（FSSplashAdDelegate）
- 六、插屏广告
 - 6.1 使用说明
 - 6.2 创建广告对象、请求广告
 - 6.3 展示广告
 - 6.4 展示时机
 - 6.5 回调说明（FSInterstitialAdDelegate）
 - 6.6 注意事项
- 七、信息流自渲染广告
 - 7.1 简介
 - 7.2 使用说明
 - 7.3 创建广告对象、请求广告
 - 7.4 回调说明
 - 7.5 辅助视图（FSNativeAdRelatedView）
 - 7.6 注意事项
- 八、信息流模板广告
 - 8.1 使用说明
 - 8.2 创建广告对象、请求广告
 - 8.3 展示广告
 - 8.4 回调说明（FSNativeExpressFeedsAdDelegate）
- 九、激励视频广告
 - 9.1 使用说明
 - 9.2 创建广告对象、请求广告
 - 9.3 展示广告
 - 9.4 展示时机
 - 9.5 回调说明（FSRewardedAdDelegate）

一、概述

FSUnionAdSDK 是一套 iOS 广告 SDK，支持开屏、插屏、激励视频、信息流自渲染、信息流模板等多种广告形态。本文档介绍 SDK 的集成方式、初始化配置及各广告类型的接入流程。

二、SDK 集成

2.1 开发环境

请确保开发及运行环境满足以下要求：

- 开发工具：推荐 **Xcode 15.4** 及以上版本
- 开发语言：**Swift 5.0、Objective-C**
- 最低部署版本：**iOS 12** 及以上
- 支持架构：真机 **arm64**；模拟器 **arm64 / x86_64**

2.2 集成方式

方式一：CocoaPods

在 Podfile 中添加以下依赖：

```
pod 'FSUnionAdSDK'
```

执行 `pod install` 后即可完成集成。最新版本请前往 [GitHub 仓库查看](#)：[iosfissionsdk](#)

方式二：手动集成

将以下文件拖入工程：

- 1. FSUnionAdSDK.framework
- 2. FSUnionAdSDK.bundle

2.3 注意事项

- 1. 纯 Objective-C 工程兼容
FSUnionAdSDK 基于 Swift 开发。若工程中没有 Swift 文件，集成后可能出现如下编译错误：

✖

Undefined symbol: __swift_FORCE_LOAD_\$_swiftCompatibility50

✖

Undefined symbol: __swift_FORCE_LOAD_\$_swiftCompatibility51

✖

Undefined symbol: __swift_FORCE_LOAD_\$_swiftCompatibility56

✖

Undefined symbol: __swift_FORCE_LOAD_\$_swiftCompatibilityConcurrency

✖

Undefined symbol: __swift_FORCE_LOAD_\$_swiftCompatibilityDynamicReplacements

✖

Linker command failed with exit code 1 (use -v to see invocation)

✖

Activity Log Complete 2024/10/15, 14:26 0.9 seconds

✖

6 errors, 18 warnings

解决方案：在工程中新建一个空的 Swift 文件，按提示创建 Bridging Header 即可。

- 1. Linker Flags
手动集成时，请在 Xcode **Build Settings** → **Other Linker Flags** 中添加 `-ObjC`。

Linking - General

Setting

FSUnionAdSDK

Other Linker Flags

-ObjC -framework "CFNetwork" -framework "CoreFoundation" -framework "CoreGraphics" -framework "FSAdEn

Quote Linker Arguments

Yes ↕

- 1. Xcode 版本兼容（SDK 1.0.7.0+）
SDK 1.0.7.0 起使用 Xcode 16 打包。若使用较新版本 Xcode 编译，且 **Other Linker Flags** 中包含 `-ld64` 或 `-ld_classic`，可能出现如下报错：

ld: Assertion failed: (it != _dylibToOrdinal.end()), function dylibToOrdinal, file OutputFile.cpp, line 5196.

clang: error: linker command failed with exit code 1 (use -v to see invocation)

✖

Assertion failed: (it != _dylibToOrdinal.end()), function dylibToOrdinal, file OutputFile.cpp, line 5196.

✖

Linker command failed with exit code 1 (use -v to see invocation)

✔

Copy rev.txt 0.1 seconds

解决方案：移除 `-ld64` 和 `-ld_classic` 链接标志。

三、SDK 初始化及全局配置

3.1 说明

- **FSAdSDKConfiguration**: SDK 全局配置单例，用于设置应用信息及运行参数。
- **appId**、**appToken** 为必填项，未正确配置将导致初始化失败；其余参数为选填。
- **FSAdSDKManager**: 负责 SDK 初始化，读取 **FSAdSDKConfiguration** 中的配置完成启动。

3.2 接口说明

```

public class FSAdSDKConfiguration: NSObject {
    ///
    public static let configuration = FSAdSDKConfiguration()

    public var debug: Bool = false

    /// ID
    public var appID: String?

    /// Token
    public var appToken: String?

    ///
    public var appName: String?

    /// CAID xid
    public var xids: [FSXIDModel]?

    /// ID
    public var customDeviceID: String?

    ///
    public var extraUserData: [String: Any]?

    /// false
    public var personalRecommend: Bool = false

    /// false
    public var teenagerModel = false

    /// "AppStore"
    public var channel: String = "AppStore"

    /// false
    public var supportWXApi: Bool = false

    /// Logo
    public var customUnionLogo: UIImage?

    private override init() {}
}

public class FSAdSDKManager: NSObject {
    /// SDK
    public static func startWithCompletionHandler(_ completionHandler: ((Bool) -> Void)?)
}

```

3.3 接入示例

建议在 AppDelegate 的 `application(_:didFinishLaunchingWithOptions:)` 中完成 SDK 初始化。配置正确的 **appID** 和 **appToken** 后，初始化回调会立即返回；SDK 内部模块在子线程异步加载。

Swift

```
import UIKit
import FSUnionAdSDK

@main
class AppDelegate: UIResponder, UIApplicationDelegate {

    var window: UIWindow?

    func application(_ application: UIApplication, didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?) -> Bool {

        // FSAdSDKManager.logEnable(true) //

        let configuration = FSAdSDKConfiguration.configuration
        configuration.appID = "xxx"
        configuration.appToken = "xxx"
        FSAdSDKManager.startWithCompletionHandler { success in
            if !success {
                print(" appID / appToken ")
            }
        }

        return true
    }
}
```

Objective-C

```
#import "AppDelegate.h"
#import <FSUnionAdSDK/FSUnionAdSDK-Swift.h>
// @import FSUnionAdSDK;

@implementation AppDelegate

- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(NSDictionary *)launchOptions {

    // [FSAdSDKManager logEnable:YES]; //

    FSAdSDKConfiguration *configuration = FSAdSDKConfiguration.configuration;
    configuration.appID = @"XXX";
    configuration.appToken = @"XXX";
    [FSAdSDKManager startWithCompletionHandler:^(BOOL success) {
        if (!success) {
            NSLog(@" appID / appToken ");
        }
    }];

    return YES;
}

@end
```

四、广告位与广告对象公共方法

4.1 广告位对象（FSAdSlot）

FSAdSlot 用于描述广告请求的代码位及通用参数。创建时只需传入代码位 ID，**无需**手动指定广告类型。广告对象的创建与参数配置均依赖此对象，详见 [4.2 广告对象创建](#)。

字段	是否必填	类型	默认值	说明
slotID	是	String	—	代码位 ID
timeout	否	Double	5	请求超时时间（秒）
requestID	否	String	—	自定义请求标识

count	否	UInt	1	请求广告数量，自渲染场景可调整
-------	---	------	---	-----------------

```
FSAdSlot *slot = [[FSAdSlot alloc] initWithSlotID:@"XXX"];
slot.timeout = 5.0;
```

FSAdSlot 还支持通过 `addRequestParam(key:value:)`、`setParam(key:value:)` 等方法设置额外请求参数（如落地页样式等），详见各广告类型章节。

4.2 广告对象创建

各类型广告对象均通过 `init(slot:)` / `initWithSlot:` 传入 **FSAdSlot** 对象进行初始化，**不支持**直接传入代码位 ID 字符串。

初始化方法	参数类型	说明
<code>init(slot:)</code> / <code>initWithSlot:</code>	FSAdSlot	传入已配置好的广告位对象

典型接入流程：

1. 创建并配置 **FSAdSlot**（设置代码位 ID、timeout 等参数）
2. 创建具体广告对象，将 **FSAdSlot** 传入 `initWithSlot:`

```
let slot = FSAdSlot(slotID: "XXX")
slot.timeout = 5.0
let ad = FSInterstitialAd(slot: slot) // slot "XXX"
```

```
FSAdSlot *slot = [[FSAdSlot alloc] initWithSlotID:@"XXX"];
slot.timeout = 5.0;
FSInterstitialAd *ad = [[FSInterstitialAd alloc] initWithSlot:slot];
```

4.3 广告类型说明

广告类型由**广告对象类**决定，SDK 在初始化时自动设置，接入方无需传入 `type` 参数。

广告对象类	广告类型
FSSplashAd	开屏
FSInterstitialAd	插屏
FSRewardedAd	激励视频
FSNativeAdsManager	信息流自渲染
FSNativeExpressFeedsAd	信息流模板

4.4 广告对象公共方法（FSUnionAd）

开屏、插屏、激励视频、自渲染等广告对象均继承自 **FSUnionAd**。模板广告 **FSNativeExpressFeedsAd** 不直接继承 **FSUnionAd**，但其内部的 **FSNativeAd** 具备相同能力。

4.4.1 公共方法

方法名	说明
<code>loadAdData()</code>	请求广告数据，各子类实现具体加载逻辑
<code>win()</code>	竞价胜出上报（ 竞价场景下竞价时必须调 ）
<code>lose(winPrice:)</code>	竞价失败上报，传入竞价方价格（ 竞价场景下竞价时必须调 ）
<code>lose(winPrice:extraWinInfo:)</code>	竞价失败上报，传入竞价方价格及额外信息；建议通过 <code>FSBiddingExtraInfoKey.loseReason</code> 传入失败原因（见 4.4.3）
<code>price()</code>	获取广告价格，单位：分；广告加载成功后可用

`extraWinInfo` 常用 key（请使用 `FSBiddingExtraInfoKey` 公开常量，避免硬编码字符串）：

常量（Swift / OC）	字符串值	说明
----------------	------	----

FSBiddingExtraInfoKey.loseReason	lose_reason	竞价失败原因, 值为 FSBiddingLoseReason 的 rawValue (Int)
FSBiddingExtraInfoKey.winSDKType	win_sdk_type	竞胜方 SDK 类型, 值为 FSBiddingWinSDKType 的 rawValue (Int, 见 4.4.4)
FSBiddingExtraInfoKey.winTitle	win_title	竞胜广告标题 (可选)
FSBiddingExtraInfoKey.winPackageName	win_package_name	竞胜广告包名 (可选)

4.4.2 使用示例

```
let slot = FSAdSlot(slotID: "XXX")
let ad = FSInterstitialAd(slot: slot)
ad.loadAdData()

///
func fs_interstitialAdSuccessToLoadAd(_ ad: FSInterstitialAd) {
    ///
    let price = ad.price()
    ///
    ad.win()
    /// SDK
    /// ad.lose(winPrice: price, extraWinInfo: [
    ///     FSBiddingExtraInfoKey.loseReason: FSBiddingLoseReason.lowPrice.rawValue,
    ///     FSBiddingExtraInfoKey.winSDKType: FSBiddingWinSDKType.pangle.rawValue
    /// ])
}
```

```
FSAdSlot *slot = [[FSAdSlot alloc] initWithSlotID:@"XXX"];
FSInterstitialAd *ad = [[FSInterstitialAd alloc] initWithSlot:slot];
[ad loadAdData];

///
- (void)fs_interstitialAdSuccessToLoadAd:(FSInterstitialAd *)ad {
    ///
    NSString *price = [ad price];
    ///
    [ad win];
    /// SDK
    /// [ad loseWithWinPrice:@"100" extraWinInfo:@{
    ///     FSBiddingExtraInfoKey.loseReason: @(FSBiddingLoseReasonLowPrice),
    ///     FSBiddingExtraInfoKey.winSDKType: @(FSBiddingWinSDKTypePangle)
    /// }];
}
```

4.4.3 FSBiddingLoseReason 竞价失败原因

通过 lose(winPrice:extraWinInfo:) 的 extraWinInfo 传入, key 使用 FSBiddingExtraInfoKey.loseReason (字符串值 lose_reason)

枚举值 (Swift)	枚举值 (OC)	rawValue	说明
lowPrice	FSBiddingLoseReasonLowPrice	1	竞价失败 (价低)
timeOut	FSBiddingLoseReasonTimeOut	2	竞价超时
floorFilter	FSBiddingLoseReasonFloorFilter	3	底价过滤
other	FSBiddingLoseReasonOther	10001	其他

4.4.4 FSBiddingWinSDKType 竞胜方 SDK 类型

通过 lose(winPrice:extraWinInfo:) 的 extraWinInfo 传入, key 使用 FSBiddingExtraInfoKey.winSDKType (字符串值 win_sdk_type) :

枚举值 (Swift)	枚举值 (OC)	rawValue	说明
pangle	FSBiddingWinSDKTypePangle	1	穿山甲
baidu	FSBiddingWinSDKTypeBaidu	2	百度
gdt	FSBiddingWinSDKTypeGdt	4	广点通
adx	FSBiddingWinSDKTypeAdx	6	ADX

未传或无法识别时，内部按 0 处理。也可继续传入字符串（如 @"1"），与枚举 rawValue 等价。

4.4.5 FSAdInteractionType 枚举

广告跳转相关回调中的 interactionType 参数含义如下：

枚举值	说明
unspecified	未定义
redirect	落地页
download	下载
deeplink	Deeplink
wxMiniApp	微信小程序
market	应用市场
justShow	纯展示

五、开屏广告

5.1 使用说明

使用 **FSSplashAd** 调用 loadAdData() 请求广告，调用 showSplashIn:bottomView: 展示广告。通过 **FSSplashAdDelegate** 接收加载、展示、点击、关闭等回调。

5.2 创建广告对象、请求广告

创建 **FSSplashAd** 时需传入 **FSAdSlot** 对象。广告位参数说明见 [4.1](#)。

```
FSAdSlot *slot = [[FSAdSlot alloc] initWithSlotID:@"XXX"];
slot.timeout = 5.0;

FSSplashAd *ad = [[FSSplashAd alloc] initWithSlot:slot];
ad.delegate = self;
[ad loadAdData];

// ad
self.splashAd = ad;
```

5.3 展示广告

开屏广告支持 **全屏** 和 **非全屏** 两种样式：

全屏(常规样式)



非全屏(常规样式)



调用 `showSplashIn:bottomView`：展示广告。`rootViewController` 用于承载广告视图及落地页跳转。

- **bottomView** 为可选参数。传入自定义 `bottomView` 时展示非全屏样式；不传则默认全屏。
- **bottomView** 高度默认为屏幕高度的 15%，可通过设置 `frame.size.height` 自定义（支持范围为屏幕高度的 15% ~ 20%），其余 `frame` 属性会被忽略。

```
[self.splashAd showSplashIn:self.rootViewController bottomView:self.bottomView];
```

5.4 展示时机

广告请求成功（fs_splashAdLoadSuccess）后即可展示，**建议**在物料下载完成（fs_splashAdMaterialDownloadSuccess）后再展示，以确保展示效果：

```
- (void)fs_splashAdMaterialDownloadSuccess:(FSSplashAd *)ad {
    [self.splashAd showSplashIn:self.rootViewController bottomView:self.bottomView];
}
```

5.5 回调说明（FSSplashAdDelegate）

回调方法	说明	备注
fs_splashAdLoadSuccess(_:)	广告请求成功	可通过 [ad price] 获取价格
fs_splashAdLoadFailed(_:error:)	广告请求失败	
fs_splashAdMaterialDownloadSuccess(_:)	物料下载成功	建议在此回调中展示广告
fs_splashAdMaterialDownloadFailed(_:error:)	物料下载失败	
fs_splashAdWillPresent(_:)	广告即将展示	
fs_splashAdDidVisible(_:)	广告曝光	
fs_splashAdFailedToPresent(_:error:)	广告展示失败	
fs_splashAdDidClick(_:)	广告点击	
fs_splashAdDidClose(_:closeType:)	广告关闭	closeType: clickSkip 跳过 / countdownToZero 倒计时结束 / click Jump 点击跳转
fs_splashAdDidCloseOtherController(_:interactionType:)	落地页 / 应用市场等页面关闭	interactionType 含义见 4.4.3

六、插屏广告

6.1 使用说明

使用 **FSInterstitialAd** 调用 loadAdData() 请求广告，调用 showAdFromRootViewController(_:) 展示广告。通过 **FSInterstitialAdDelegate** 接收各阶段回调。

6.2 创建广告对象、请求广告

```
FSAdSlot *slot = [[FSAdSlot alloc] initWithSlotID:@"XXX"];
slot.timeout = 5.0;

FSInterstitialAd *ad = [[FSInterstitialAd alloc] initWithSlot:slot];
ad.videoMuted = YES; //
ad.delegate = self;
[ad loadAdData];

self.ad = ad;
```

6.3 展示广告

调用 showAdFromRootViewController(_:) 展示广告，传入当前用于承载广告的 ViewController：

```
ad.showAdFromRootViewController(self)
```

6.4 展示时机

广告请求成功（fs_interstitialAdSuccessToLoadAd）后即可展示，**建议**在物料下载完成（fs_interstitialAdSuccessToDwonlaodMaterial）后再展示：

```
func fs_interstitialAdSuccessToDwonloadMaterial(_ ad: FSInterstitialAd) {
    ad.showAdFromRootViewController(self)
}
```

6.5 回调说明（FSInterstitialAdDelegate）

回调方法	说明	备注
fs_interstitialAdSuccessToLoadAd(_:)	广告请求成功	可通过 ad.price() 获取价格
fs_interstitialAdFailedToLoadAd(_:error:)	广告请求失败	
fs_interstitialAdSuccessToDwonloadMaterial(_:)	物料下载成功	建议在此回调中展示广告
fs_interstitialAdFailedToDwonloadMaterial(_:error:)	物料下载失败	
fs_interstitialAdRenderSuccess(_:)	渲染成功	
fs_interstitialAdRenderFail(_:error:)	渲染失败	
fs_interstitialAdWillPresent(_:)	广告即将展示	
fs_interstitialAdDidVisible(_:)	广告曝光	
fs_interstitialAdDidClick(_:)	广告点击	
fs_interstitialAdDidClose(_:)	广告关闭	v1.0.1.7 起插屏不再自动关闭
fs_interstitialAdDidCloseOtherController(_:interactionType:)	落地页 / 应用市场等页面关闭	interactionType 含义见 4.4.3

6.6 注意事项

- 展示时需传入有效的 rootViewController，用于广告展示及落地页跳转。
- 插屏广告通过 present 方式展示，请确保传入的 ViewController **未被其他 Modal 占用**，否则可能因 presentedViewController 已存在而导致展示失败。

七、信息流自渲染广告

7.1 简介

自渲染广告允许开发者自行定义广告布局和展示样式，SDK 提供广告素材及交互能力，由接入方完成 UI 渲染。

7.2 使用说明

使用 **FSNativeAdsManager** 调用 loadAd() 请求广告，通过 **FSNativeAdsManagerDelegate** 获取广告列表，通过 **FSNativeAdDelegate** 监听曝光、点击等事件。

7.3 创建广告对象、请求广告

```
FSAdSlot *slot = [[FSAdSlot alloc] initWithSlotID:@"XXX"];
FSNativeAdsManager *loadManager = [[FSNativeAdsManager alloc] initWithSlot:slot];
loadManager.delegate = self;
[loadManager loadAd];
self.loadManager = loadManager;
```

```
- (void)fs_nativeAdsManagerSuccessToLoad:(FSNativeAdsManager *)adsManager
    nativeAds:(NSArray<FSNativeAd *> *)nativeAds {
    if (nativeAds.count == 0) { return; }
    self.nativeAd = nativeAds.firstObject;
}

- (void)fs_nativeAdsManagerDidFail:(FSNativeAdsManager *)adsManager error:(NSError *)error {
    NSLog(@"": %@", error);
}
```

7.4 回调说明

FSNativeAdsManagerDelegate

回调方法	说明	备注
fs_nativeAdsManagerSuccessToLoad(_:nativeAds:)	加载成功	可通过 nativeAd.price() 获取价格
fs_nativeAdsManagerDidFail(_:error:)	加载失败	

FSNativeAdDelegate

回调方法	说明
fs_nativeAdWillVisible(nativeAd:)	广告即将曝光
fs_nativeAdDidBecomeVisible(nativeAd:)	广告有效曝光
fs_nativeAdDidClick(nativeAd:containerView:)	广告点击
fs_nativeAdDidCloseOtherController(nativeAd:interactionType:)	落地页 / 应用市场等页面关闭

7.5 辅助视图（FSNativeAdRelatedView）

通过 **FSNativeAdRelatedView** 可获取 Logo、视频播放器等预制视图：

API	说明	备注
logoImageView	Logo 图片视图	
videoAdView	视频播放器视图	
refreshData(_:)	绑定广告数据	使用前必须先调用

监听视频播放状态，需设置 videoAdView 的 **FSVideoAdViewDelegate**：

```
self.nativeAdRelatedView.videoAdView.delegate = self;
```

FSVideoAdViewDelegate

回调方法	说明
fs_videoAdViewReadyToPlay(_:)	播放器就绪，可以播放
fs_videoAdViewDidPlayFinish(_:)	播放完成
fs_videoAdView(_:didLoadFailedWithError:)	加载失败
fs_videoAdView(_:playStateDidChange:)	播放状态变更

7.6 注意事项

- 1. 获取广告素材并完成 UI 布局后，需调用 registerContainer(_:withClickableViews:) 注册容器和可点击区域，并调用 refreshData(_:) 刷新数据。
- 2. 每次更新广告素材后，需重新调用 refreshData(_:)。
- 3. 视频宽高可通过 nativeAd.material.videoWidth / videoHeight 获取，在 fs_videoAdViewReadyToPlay 回调后生效，否则为 0。

八、信息流模板广告

8.1 使用说明

使用 **FSNativeExpressFeedsAd** 调用 loadAdData() 请求广告，调用 showInView(_:) 展示广告。通过 **FSNativeExpressFeedsAdDelegate** 接收各阶段回调。

模板类型	expressType 枚举值	说明
小图 Banner	.smallBanner	具体样式请参考 Demo
大图 Banner	.banner	宽高比固定: height = width × 9/16 + 108，需提前设置容器尺寸
全屏模板	.full	
中图 Banner	.midBanner	

8.2 创建广告对象、请求广告

```
FSAdSlot *slot = [[FSAdSlot alloc] initWithSlotID:@"XXX"];

//
[slot addRequestParamWithKey:FSAdParamsKey.landStyle value:@(FSAdLandStyleSheet)];
[slot addRequestParamWithKey:FSAdParamsKey.landHeight value:@(0)];
[slot addRequestParamWithKey:FSAdParamsKey.landOutsideCancel value:@(NO)];
```

落地页附加参数说明：

参数	类型	默认值	说明
landStyle	FSAdLandStyle	.none	.none 全屏；.sheet 半屏弹出
landHeight	CGFloat	0	半屏高度，0 为默认半屏高度
landOutsideCancel	Bool	NO	点击空白区域是否关闭落地页

```
FSNativeExpressFeedsAd *ad = [[FSNativeExpressFeedsAd alloc] initWithSlot:slot];
ad.expressType = FSNativeExpressTypeSmallBanner;
ad.delegate = self;
ad.rootViewController = self; //
[ad loadAdData];

self.feedBannerAd = ad;
```

8.3 展示广告

在 fs_expressFeedAdLoadSuccess 回调成功后调用 showInView(_):

```
[self.feedBannerAd showInView:self.adCell.contentView];
```

8.4 回调说明（FSNativeExpressFeedsAdDelegate）

回调方法	说明	备注
fs_expressFeedAdLoadSuccess(_:)	广告加载成功	此后方可调用 showInView(_:); 可通过 ad.price() 获取价格 (单位: 分)
fs_expressFeedAdLoadFailed(_:withError:)	广告加载失败	
fs_expressFeedAdDidVisible(_:)	广告曝光	
fs_expressFeedAdPresentFailed(_:withError:)	广告展示失败	
fs_expressFeedAdDidClosed(_:)	广告关闭	
fs_expressFeedAdDidClicked(_:)	广告点击	
fs_expressFeedAdDidCloseOtherController(_: interactionType:)	落地页 / 应用市场等页面关闭	interactionType 含义见 4.4.3

九、激励视频广告

9.1 使用说明

使用 **FSRewardedAd** 调用 loadAdData() 请求广告，调用 showAdFromRootViewController(_:) 展示广告。通过 **FSRewardedAdDelegate** 接收加载、展示、奖励发放等回调。

9.2 创建广告对象、请求广告

```
FSAdSlot *slot = [[FSAdSlot alloc] initWithSlotID:@"XXX"];

FSRewardedAd *rewardedAd = [[FSRewardedAd alloc] initWithSlot:slot];
rewardedAd.delegate = self;
[rewardedAd loadAdData];

self.rewardedAd = rewardedAd;
```

9.3 展示广告

```
ad.showAdFromRootViewController(self)
```

9.4 展示时机

广告请求成功（fs_rewardedAdDidLoadSuccess）后即可展示，**建议**在物料下载完成（fs_rewardedAdDidDownloadMaterial）后再展示：

```
- (void)fs_rewardedAdDidDownloadMaterial:(FSRewardedAd *)ad {
    [self.rewardedAd showAdFromRootViewController:self];
}
```

9.5 回调说明（FSRewardedAdDelegate）

回调方法	说明	备注
fs_rewardedAdDidLoadSuccess(_:)	广告请求成功	可通过 ad.price() 获取价格
fs_rewardedAd(_:didLoadFailedWithError:)	广告请求失败	
fs_rewardedAdDidDownloadMaterial(_:)	物料下载成功	建议在此回调中展示广告
fs_rewardedAd(_:didRenderFailedWithError:)	渲染失败	
fs_rewardedAdWillPresent(_:)	广告即将展示	
fs_rewardedAdDidVisible(_:)	广告曝光	
fs_rewardedAdDidSucceed(_:)	奖励发放成功	
fs_rewardedAdDidClose(_:)	广告关闭	
fs_rewardedAdDidClick(_:)	广告点击	
fs_rewardedAdDidSkip(_:)	用户跳过广告	
fs_rewardedAdVideoDidPlayFinish(_:)	视频播放完成	
fs_rewardedAdDidCloseOtherController(_:interactionType:)	落地页 / 应用市场等页面关闭	interactionType 含义见 4.4.3